

HMS Project: Patient Appointment Management Module

Sandhan Meher

18 July 2025

Enabling Seamless Patient-Doctor Connections

Project & Module Context

- **Hospital Management System (HMS):**

- A comprehensive digital platform to manage hospital operations seamlessly.

- **Importance of This Module:**

- Direct interface for patients.
- Improves patient experience via self-service booking.
- Automates and streamlines scheduling operations.

- **My Role:**

- Focused on patient-side appointment booking, management, and history.

Module Overview – What It Does

Purpose & Core Functionalities with Method References

- **Purpose:**

- Enable patients to:
 - **Book new appointments** with doctors.
 - (e.g., `DoctorAvailabilityController.bookAppointment()`)
 - **View upcoming and past appointments.**
 - (e.g.,
`RecipientAppointmentController.getUpcomingAppointments()`,
`RecipientAppointmentController.getPastAppointments()`)
 - **Cancel appointments** when needed.
 - (e.g.,
`RecipientAppointmentController.cancelAppointment()`)

- **Core Functionalities:**

- **Doctor Availability Discovery:**
 - (e.g.,
`DoctorAvailabilityController.loadAvailableSlots()`,
`getGroupedAvailabilityList()`)
- **Appointment Booking Workflow:**
 - (e.g., `DoctorAvailabilityController.bookAppointment()`)
- **History & Status Filters:**
 - (e.g.,
`RecipientAppointmentController.updateFilteredAppointments()`)
- **Email Notifications:**
 - (Utilizes `MailSend.sendEmail()`)

[Placeholder: Patient Journey Flow: Discover → Book → View →
Manage]

Key Features – Appointment Booking

Streamlined Workflow & Robust Validation

- **Doctor Availability Display:**

- Organized **date-wise** for easy Browse.
- **Flatpickr** integration for intuitive date selection.
- Clear **Morning, Afternoon, Evening** slot categorization.

- **Booking Process Highlights:**

- Simple **slot selection** initiates the booking.
- Triggers comprehensive **backend validation**.
- Leads to immediate **confirmation** or feedback.

- **Key Backend Validations:**

- Prevents booking in the **past**.
- Validates **doctor and recipient status** (e.g., active, not suspended).
- Ensures **no double-booking** or time overlap for the patient.
- Checks **slot capacity** and adherence to doctor's **working hours**.
- Enforces any required **notice period** before booking.

- **User Feedback:**

- Smooth **loading animation** displayed during the booking process.
- Clear **success or failure messages** for user clarity.

Key Features – Appointment History & Management

Comprehensive Overview & Control for Patients

- **Comprehensive History View:**

- **Future Appointments:** Displays upcoming appointments, including those that are Upcoming, Pending, Booked, or recently Cancelled.
- **Past Appointments:** Shows historical records of Completed and Cancelled appointments.

- **Flexible Filtering Options:**

- **Time Filter:** Easily switch between **Future** and **Past** appointment views.
- **Status Filter:** Filter appointments by specific states: All, Pending, Booked, Completed, and Cancelled.

- **Efficient Pagination:**

- Seamlessly handles **long lists of appointments**, improving user experience and performance.

- **Appointment Cancellation:**

- Patients can **cancel only eligible future appointments** (typically Pending or Booked status).
- Cancellation requests are **strictly validated on the backend** to adhere to business rules.

- **Automated Email Notifications:**

- Confirmation emails are automatically sent to the patient upon **successful booking** and **cancellation**.

Technical Stack & Architecture

Foundation of the Module

- **Frontend:**

- **JSF + JSP:** Leveraging a component-based approach for the user interface.
- **Tailwind CSS:** Ensuring a clean, modern, and responsive design.
- **JavaScript:** Enhancing user interactions for date and slot selection.

- **Backend (Java):**

- **Controllers:** `DoctorAvailabilityController` and `RecipientAppointmentController` manage business logic and UI interactions.
- **DAO Layer:** Data Access Objects (e.g., `AppointmentDaoImpl`) for robust database operations.
- **Hibernate:** Utilized as the Object-Relational Mapping (ORM) framework for efficient database interaction.
- **JavaMail:** Integrated for sending automated email notifications.
- **Log4j:** Implemented for comprehensive logging, debugging, and issue tracking.

- **Database:**

- **MySQL:** Serving as the relational database for storing all application data.

- **Design Philosophy:**

- **MVC Architecture:** Following the Model-View-Controller pattern for a clean separation of layers and maintainable code.

[Placeholder: Technology Logos / MVC Architecture Diagram]

Progress & Achievements So Far

Key Milestones Achieved

- ✓ **Completed Booking and Cancellation Flow:**
 - Full end-to-end process for patients to book and cancel appointments seamlessly.
- ✓ **Comprehensive Appointment Filters:**
 - Implemented filters by **time** (Future/Past) and **status** (All, Pending, Booked, Completed, Cancelled).
- ✓ **Robust Backend Validation and Error Handling:**
 - Extensive validation logic ensures data integrity and adherence to business rules.
 - Graceful handling of errors for a smooth user experience.
- ✓ **Responsive UI with Real-time Feedback:**
 - User interface adapts to different screen sizes, providing real-time visual feedback (e.g., loading animations).
- ✓ **Email Notifications Integrated:**
 - Automated email confirmations sent for booking and cancellation actions.
- ✓ **Modular and Maintainable Structure:**
 - Developed with clear separation of concerns (MVC, DAO layer) for easy future expansion and maintenance.

Future Enhancements & Next Steps

Roadmap for Continued Module Evolution

- **Immediate Priority:**

- **User Session Integration:** Crucially, replace hardcoded recipient IDs with dynamic, session-based IDs from the logged-in user.

- **Planned Features:**

- **Modify Appointment (Reschedule Option):** Enable patients to easily reschedule existing appointments.
- **More Intuitive Doctor Selection UI:** Develop a user-friendly interface for patients to browse and select doctors.
- **UI/UX Polish:** Continuous refinement of the user interface and experience based on feedback.

- **Testing Quality Assurance:**

- Implement comprehensive **Unit, Integration, and Performance testing** to ensure robustness and scalability.

[Placeholder: Roadmap Graphic / Future Features Icons]